



Saar la carte

A Restaurant Simulation



Contents

1	Organizational Matters	1
1.1	Design	1
1.2	Implementation	3
1.3	Tools Needed	4
1.4	Submissions	5
1.5	Interaction with Tutors regarding Task Specification	6
2	Saar la Carte	8
2.1	Introduction	8
2.2	Simulation Resources	10
2.3	Simulation Details	27
2.3.1	Logging	28
2.3.2	Initialization	28
2.3.3	Simulation Statistics	38
3	Technical Details	40
3.1	Command-Line Interface	40
3.2	Build Script	40
3.3	Code Quality	41
4	Tests	42
4.1	Unit Tests	42
4.2	Integration Tests	42
4.3	System Tests	43
A	Appendix	44
A.1	Example recipes and restaurants	44
A.2	Features	46

1. Organizational Matters

This document covers the content-related aspects of the group phase of the Software Engineering Lab. You will be assigned a group in the CMS.



Please note:

For the schedule and the exam modalities of the group phase, please additionally note the information from the document on Organizational Matters, which has already been provided to you in the CMS at the beginning of the SE Lab.

Like customers in real life, the Chair is volatile in its requirements for the project to be implemented. Therefore, there will be a change to the specification after the design review. Your design must be kept flexible enough to accommodate the changes in the task without much effort. The specific changes will then be announced in a separate document after the design reviews.

1.1. Design

During the design phase, your group must complete the following tasks:

1. You must create a UML class diagram of your system. The class diagram must contain all relevant classes of your implementation, as well as their most important properties and methods. This should show which data and which functionality you encapsulate in each class. The relations of the classes among themselves must be modeled correctly. In the class diagram, all classes as well as all non-trivial methods together with their parameters must be specified. Trivial methods include, in particular, simple get or set functions for a class' own attributes, hashCode, equals, and toString.
2. You need to model the simulation in a state diagram to better understand the simulation flow and to ensure that you also fully consider this simulation flow in your design. Create a state diagram that is beginning at the start of the simulation (i.e., the configuration files are not yet loaded) and models the simulation itself up to the end of the simulation. Try to make the diagram as fine-granular and detailed as possible.

3. You need to demonstrate the interaction of your classes using UML sequence diagrams for the following scenarios. When the requirements change after the design review, we will announce a third scenario for which you will need to demonstrate the interaction of your classes using another UML sequence diagram.

- Initialization of the simulation, with ($\text{maxTicks} = 0$), until the program terminates. Initialize one object of every type (e.g., ingredient, recipe with and without basic dish, restaurant, table per table type, customer per customer type, incident per incident type).

- Handling the preparation phase and the first tick of the serving phase of evening 30 for the restaurant *Fire*. ~~The restaurant's pantry is completely empty.~~ The pantry was completely restocked the evening before, there's leftover 2000 g chicken, 1000 g rice, 5000 mL oil. This evening, the restaurant expects a REGULAR customer group with id 3 that was there once, then all members ordered potato soup. Additionally, there are three CASUAL groups. The details are provided as as a single JSON file in Listing A.1.

- Handling tick 6 of evening 35 for the restaurant FIRE, which now has 2 waiters. You can assume that the pantry was stocked with ingredients for 50 chicken rice, 50 beef pasta and 50 potato soup. You can assume that group 1 has already arrived, was seated at table 3 (merged with table 4 and 5), ordered 2 chicken rice and 6 potato soup in tick 1 and was served the potato soup in tick 3 and chicken rice in 4. There is an additional customer group 6, that arrived in tick 4, that was seated at table 2 (merged with table 6), and that all ordered beef pasta. The details are provided in Listing A.1 and additionally for the additional customer group are provided in Listing A.2.

4. Every group member must read and understand the complete specification and the group design.

5. You must draw up a detailed implementation plan using the mandatory template provided in the CMS and submit it in your GITLAB repository.

The plan must define responsibilities for each feature (cf. Appendix Table 1 and Table 2), specifying one primarily responsible group member for its functionality and correctness in the implementation, as well as two other group members responsible for testing it. Responsibilities must be divided fairly and documented in the implementation plan. For features F01–F07, each group member must be primarily responsible for 1–2 features.

The implementation plan should clearly identify dependencies between features, establish the order of milestones, and include a detailed time and work schedule.

1.2. Implementation

In this phase, you will complete the actual implementation of the simulator in your group. You will also write unit tests, integration tests, and system tests and create an implementation report. Your simulator implementation must meet the following requirements:

1. Your code must be well structured, well documented and follow the concepts presented in the lecture. We will examine your implementation with the code analysis tool DETEKT. This happens automatically; your project needs to *build* and pass the code analysis tool on your `main` branch before our tests can be run on your code. (If there are some rules of the code analysis tool which appear to you to be unreasonable in a specific situation, please contact your tutor. We expect you to be able to provide sufficient justification why you think that the rule is unreasonable in this situation.)

With the build script we have created for you, you can run this tool yourself from the beginning of the implementation phase to check if your code still has problems.

In addition, we will manually examine your code to see if you followed higher-level concepts presented in the lecture.

2. Your implementation must correctly implement the specification, for which your implementation must pass the system tests we create. We run our system tests on your implementation regularly during the implementation phase, and the test results will be provided to you at regular intervals. The tests are divided into different categories. For each category, there is a specific subset that your implementation must pass. Additional tests will be provided for further debugging. We will release a separate document in which we will specify the mandatory test set.
3. You must implement unit tests, integration tests, and system tests that cover all requirements described in the specification. Your tests are intended to achieve high code coverage while testing reasonable scenarios, which is ensured by finding mutants with your system tests¹. To fulfill this criterion, your system tests must pass a specified amount of the released mutants. Additionally, your implementation must pass the unit tests, integration tests, and system tests that you create. In total, your test suite must provide a high coverage of your implementation, especially of the simulation logic.
4. Each individual group member must make a significant contribution to the group's implementation and test suite through their assigned features, covering the aspects described above. For implementation features, the group member is responsible for its functionality and correctness in the implementation. For testing features, the assigned group members are each responsible for writing and maintaining correct, suitable and sufficient tests.

The individual responsibilities and contributions of each group member must be documented in the group's implementation report. This report and an additional appendix

¹https://en.wikipedia.org/wiki/Mutation_testing

1 should attribute the assigned features as well as the contributions of each group mem-
2 ber, describing who contributed to which feature implementation, feature test suite
3 or system tests. Additionally, it must contain how and why the feature assignment
4 changed compared to the implementation plan. Templates will be provided in the
5 CMS, which are mandatory to use for the submission in your GITLAB repository.

6 1.3. Tools Needed

7 To participate in the SE Lab, you need to bring your own laptop. You need to install the
8 following tools on your device:

- 9 ■ **PlantUML², which must be used for all design artefacts**
- 10 ■ **JAVA 21** (our reference platform uses the latest LTS-version *Temurin® JDK 21.0.12*)³
- 11 ■ **Version-control system *git***⁴
- 12 ■ **IDE of your choice** (we recommend **INTELLIJ**)⁵

13 The build system GRADLE, which we use, does not need to be explicitly installed at your end,
14 because we provide you with a project in which a *Gradle wrapper* is already included, which
15 takes care of the installation by itself. Kotlin also does not have to be installed manually
16 but will be installed via Gradle. All other tools you need do not need to be installed, but are
17 already pre-configured in the project by us. All libraries included in the Gradle configuration
18 file (`build.gradle.kts`) may be used. Other libraries must not be used.

19 For this project, we do not prohibit you from using any generative tools to write code or
20 create a design. However, you are the one responsible for the quality and suitability of
21 the results and how to integrate them into the group project. You must understand your
22 contributions in detail (e.g., code, design, tests) and be able to explain and defend them
23 both to your group members and to the examiners. **It is not an appropriate contribution**
24 **to just push boilerplate code or to create very similar or pointless tests.** To reduce
25 potential conflicts within the group, we encourage each group to discuss and establish their
26 own specific internal rules and processes for using such technical tools.

27 You should state in your implementation plan and report which technical tools you used and
28 in which aspects those tools were involved.

²<https://plantuml.com>

³<https://adoptium.net/de/temurin/releases?version=21&os=any&arch=any>

⁴<https://git-scm.com/>

⁵<https://www.jetbrains.com/idea/>

1.4. Submissions

We will provide you with *git* repositories for the group phase in our GITLAB⁶. To be able to clone or push to the provided repositories, you need an SSH key. You have already learned how to generate a SSH key and which other technical requirements are necessary for using the repository in the practical tutorial.



Attention!

Be sure here to have GIT properly and consistently configured on all devices you use to commit the code with your full name so that your commits are also assigned to you in our GITLAB. We accept only one clearly identifiable user name and mail address per group member. In particular, if we cannot identify a persons contribution due to using unclear or multiple user names, we cannot include this contribution in our evaluation.

A (design) repository is provided for submitting your design. All design documents and associated diagrams must be created using PLANTUML and be pushed to this repository. The implementation plan and feature responsibilities must be based on the provided template and pushed to the repository. **For this, you must create a draft for the design review submission.** Every member of the group is responsible to ensure that the latest version of their modules' documents is uploaded in time. For the group design re-evaluation, you must also upload the current state of your design with all changes since the design defense highlighted in blue, and a written document describing and justifying the amendments you made to solve the previously detected flaws.



Attention!

All documents that you submit for the design review and design defense have to be provided in *readable, machine-written PDF* format as well as the original tool format (plantUML). For the implementation plan and report, you must use *markdown* based on the template provided in the CMS. For the feature assignment, you must use *yaml* based on the template provided in the CMS.

For every design submission, you must push and mark your submission commit with a tag⁷. To create a tag for the current commit, use the GIT command `git tag <tagname>`. This

⁶<https://sopra.se.cs.uni-saarland.de/>

⁷<https://git-scm.com/book/en/v2/Git-Basics-Tagging>

1 has to occur, at latest, 1 hour before your assigned appointment for the design review and
2 defense. Use the following tags:

- 3 ▪ for the design review: *design_review*
- 4 ▪ for the design defense: *design_defense*
- 5 ▪ for the group design re-evaluation: *design_revised*



Attention!

6 All documents that you submit for the design review and design defense have to be provided in readable *PDF* format. You must submit the implementation plan, report, and feature assignment based on the template provided in the CMS.

7 The implementation repository will contain multiple branches: A protected `results` branch,
8 which can not be worked on, as we will provide test results there. The `main` branch is to use
9 for your implementation. There, you will be provided with a project with a minimal skeleton
10 at the start of your implementation phase. We will not consider other branches than `main`
11 for the code submission.

12 For the final submission of your implementation, the *last* commit on the `main` branch made
13 to the repository *no later than 18:00 CEST* on the day of code submission counts. Your
14 submissions must be executable on our reference platform (*Debian 12 with Eclipse Temurin*
15 *21.0.12*). Your submission must include the implementation, the unit and system tests, as
16 well as the implementation report. It is your responsibility to verify that a push has arrived
17 in the repository and that everything is up to date in a timely manner.

18 1.5. Interaction with Tutors regarding Task Specification

19 Ambiguities in the task specification, which follows in the next chapters, are unavoidable. As
20 in real software projects, there may be situations which are not completely specified in the
21 specification text. Often this holds, for instance, for specific corner cases. In such a case,
22 interaction with the customer is necessary. That is, you need to interact with the tutors
23 (e.g., in the forum) to clarify unspecified details. We will also provide an update website,
24 where we will post clarifications. Please check this website and previous forum posts before
25 posting a new question in the forum.

26 We try our best to keep the task specification unambiguous, but as in real-world software
27 projects, some corner cases may not be fully covered, and interacting with the customer in
28 these kinds of situations is unavoidable. In our context, that means clarifying unspecified

- 1 details with the tutors (e.g., in the forum). We also maintain an update website with clarifica-
- 2 tions; please check it, along with earlier forum posts, before posting a new question. [https:](https://forum.se.cs.uni-saarland.de:51443/t/specification-adjustments/16)
- 3 [//forum.se.cs.uni-saarland.de:51443/t/specification-adjustments/16](https://forum.se.cs.uni-saarland.de:51443/t/specification-adjustments/16)

2. Saar la Carte

2.1. Introduction

Running a successful restaurant is a constant balance between preparation, uncertainty, and limited resources. Every evening, restaurant managers must make decisions long before the first guest arrives. They need to estimate how many customers might visit, which dishes are likely to be ordered, how many ingredients to keep in stock, and how to distribute work among their staff. While perfect planning would be desirable, restaurant managers rarely possess complete information. Instead, they rely on experience, routines, and educated guesses.

Saar la Carte models this challenge. Rather than focusing on the financial aspects of running a business, it concentrates on the operational side of restaurant management. The simulation follows a collection of restaurants over multiple evenings and explores how their staff, customers, ingredients, ~~and menus~~ **menus**, and **reputation** interact throughout a typical service period. Although inspired by real-world hospitality businesses, the simulation intentionally simplifies many aspects of restaurant operations in order to create a deterministic and understandable environment.

At the heart of every restaurant lies a simple objective: serving food to customers in a timely manner. Achieving this goal, however, requires cooperation between many different roles. Managers need to ensure that enough ingredients are available, cooks must transform those ingredients into meals, ~~and~~ waitstaff must guide customers through the dining experience, and delivery drivers may carry meals beyond the walls of the restaurant. Since all of these activities depend on one another, a delay or shortage in one area often affects the entire restaurant.

One of the greatest challenges for restaurant management is forecasting future demand. Restaurants must purchase ingredients before customers actually place their orders. Buying too little results in unavailable dishes and disappointed customers. ~~Buying too much ties up resources and creates unnecessary inventory.~~ **Buying too much creates waste when ingredients eventually expire.** Since restaurants cannot predict customer choices perfectly, they must estimate future demand based on the information available to them. Regular customers help reduce this uncertainty because their habits become known over time. **Event reservations provide even more reliable information, allowing restaurants to prepare for a large number of guests days in advance.** Restaurants can thus prepare for their expected preferences before service begins.

1 The kitchen represents the operational center of the restaurant. In professional kitchens,
2 cooks are typically organized in a hierarchy in which different positions specialize in particular
3 tasks. Experienced chefs coordinate kitchen operations, while specialists focus on specific
4 categories of dishes. This structure allows restaurants to prepare a wide variety of meals
5 while maintaining consistent quality. The simulation mirrors this organization by assigning
6 different capabilities to different cook roles. Consequently, a restaurant's ability to serve
7 a particular dish depends not only on ingredient availability but also on the skills present
8 among its kitchen staff.

9 Unlike the earlier version of the simulation, inventory management now involves the risk of
10 spoilage. Restaurant managers must consider not only how much food to purchase, but
11 also how long ingredients remain usable. Ingredients that exceed their best-before date are
12 discarded before service begins. This creates a classic trade-off between maintaining large
13 safety reserves and minimizing food waste. Restaurants that prepare aggressively may reduce
14 shortages but increase waste, whereas conservative planning may leave customers without
15 their preferred meals.

16 The front of house plays an equally important role. Customers rarely see the planning and
17 coordination taking place behind the kitchen doors, yet their dining experience is heavily influ-
18 enced by the staff serving them. The waitstaff must decide how to seat guests, which tables
19 should remain available for future arrivals, and how to distribute work among employees. In
20 real restaurants, experienced managers often avoid assigning all customers to a single waiter
21 while others remain idle. Similarly, they try to avoid situations where large groups are turned
22 away because all suitable tables have already been occupied by smaller parties. The simu-
23 lation reflects these practical considerations through its seating and workload management
24 rules. Event groups, in particular, require advance planning because large reservations can
25 significantly influence seating availability for the entire evening. Managers therefore face the
26 challenge of accommodating guaranteed future demand while still maximizing opportunities
27 for walk-in customers.

28 Modern restaurants increasingly compete not only through food quality but also through
29 accessibility. For this reason, the simulation introduces delivery services. Delivery allows
30 restaurants to serve customers who may never physically enter the establishment. However,
31 each delivery consumes additional resources and introduces travel times that may affect
32 customer satisfaction. Managers must therefore balance in-house service with delivery com-
33 mitments while ensuring that both groups receive acceptable service quality.

34 Reputation represents another important factor influencing restaurant success. Potential
35 customers frequently rely on the experiences of previous guests when selecting a dining lo-
36 cation. Positive experiences encourage future visits, while recurring problems can discourage
37 customers from choosing the restaurant. The simulation models this process through a
38 global rating service that influences future customer decisions. Consequently, the effects of
39 management decisions may persist long after a single evening has concluded.

40 Customers themselves are far from identical. Some visit regularly and develop habits that
41 restaurants learn to anticipate. Others visit only occasionally and bring a degree of unpre-

dictability to the evening, as they rely on restaurant information and ratings when making decisions. Event organizers plan large gatherings well in advance. Delivery customers prioritize convenience and accessibility. Individual guests may have favorite dishes, ingredients they particularly enjoy, or foods they prefer to avoid entirely. Together, these preferences create highly varied patterns of demand that restaurants must accommodate.

Restaurants do not operate in a static environment. Staffing can change, suppliers may alter packaging sizes, certain ingredients may become temporarily unavailable, and recipes may evolve over time. Such changes force restaurants to continuously adapt their planning and operations. Successful restaurant management therefore depends not only on performing routine tasks efficiently but also on responding effectively to unexpected changes.

Ultimately, the simulation explores how operational decisions shape customer experiences over time. Managers anticipate future demand, cooks transform resources into meals, waitstaff coordinate customer interactions, ~~and guests pursue satisfying dining experiences,~~ and drivers extend the restaurant beyond its physical location, and customers share their experiences through ratings. Together, these elements create a dynamic environment where planning, adaptability, and resource management contribute to the long-term success of a restaurant.

2.2. Simulation Resources

In the following sections, we will describe the features of the simulation, their purpose, requirements and interactions. Please note that this is only an approximation of the real world and as such might not be coherent with any restaurants and their strategies in the real world. The order of how the parts in the simulation are performed will be described in the next section.

Time and Ticks

We will perform a simulation which simulates one or multiple evenings in a restaurant, in which multiple actions and incidents happen. The evenings start from `evening = 1`, and each evening consists of a preparation phase and a serving phase. The preparation phase is simulated as a single phase per restaurant in which everything is handled consecutively. The serving phase simulates the time between 6 and 10 pm in tick-based manner, where each tick covers 10 minutes. The ticks start with number `tick = 1` and increment through the evenings. All actions within a tick are performed consecutively.

Supplier

The supplier is a global entity that is responsible for providing the restaurants with ingredients. In each preparation phase, each restaurant will procure ingredients from the supplier

1 if they require at least one ingredient, and they will procure each needed ingredient with a
2 specific amount. You may assume that, in general, the supplier has all ingredients available
3 in any quantity.

4 **Restaurant**

5 The simulation will contain one or multiple restaurants that aim to cook meals and serve
6 them to their customers. There needs to be at least one restaurant in the simulation. Each
7 restaurant is characterized by a unique `id` and a unique `name`.

8 **Type** Restaurants have different types, which determine the kind of dishes they serve and
9 set the basic dishes available in each restaurant. There are `EUROPEAN`, `ASIAN`, `AFRICAN`,
10 and `AMERICAN` restaurant types. For each restaurant type, there exists a fixed set
11 of basic dishes that the restaurant offers. There exists a default recipe for each basic
12 dish, but restaurants are allowed to customize them.

13 **Opening Hours** Restaurants may have adjusted opening hours within the set time frame,
14 which are rounded to match complete ticks. A restaurant might, for example, be open
15 from 6:40 to 9:20 pm.

16 **Delivery Drivers** A restaurant may also work with an external delivery service, which sup-
17 plies them with orders from a customer and assigns them drivers to deliver the food.
18 A restaurant has access to a fixed number of drivers.

19 **Event** Restaurants can cater to events.

20 **Ratings** The positive and negative ratings a restaurant has in the rating service at the start
21 of the simulation.

22 **Recipes** All recipes owned by the restaurant. Each recipe describes how to cook meals of a
23 given dish.

24 We model our restaurant with 2 separate subentities: kitchen with the kitchen staff and
25 front of house with the waitstaff.

26 **Kitchen**

27 The kitchen is responsible for planning and buying ingredients, cooking meals of a dish
28 according to a recipe and providing the meals to the waitstaff. In the preparation phase,
29 the kitchen staff checks the pantry and throws away any ingredient that is beyond their
30 best-before date. Then, they plan the ingredients for the upcoming evening based on
31 the reserved tables and the information they already have available. The first planning step
32 is for the known `REGULAR` groups, so those `REGULAR` groups that already have visited the
33 restaurant at least once. For those known regulars planning to visit on this evening, they
34 plan all ingredients to be able to cook all orders of the group's last three visits. In case there
35 are less than 3 visits, the kitchen considers all of those visits. For `EVENTS`, they plan all

Restaurant

- id ($\text{id} \geq 0$) The unique identifier of the restaurant.
- name (name) The unique name of the restaurant.
- type ($\text{type} \in [\text{EUROPEAN}, \text{ASIAN}, \text{AFRICAN}, \text{AMERICAN}]$) The type of the restaurant.
- openingTickStart ($\text{openingTickStart} \in [1; 24]$) The first tick the restaurant will operate for the evening.
- openingTickEnd ($\text{openingTickEnd} \in [1; 24]$
 $\wedge \text{openingTickEnd} > \text{openingTickStart}$) The last tick in which the restaurant will operate.
- deliveryDrivers ($\text{deliveryDrivers} \geq 0$) The amount of drivers the delivery service provides to this restaurant to deliver meals. In case there is no connection to a delivery service, this number is 0.
- event ($\text{event} \in \{\text{true}, \text{false}\}$) Whether the restaurant can cater to events.
- positiveRatings ($\text{positiveRatings} \geq 0$) The number of positive ratings the restaurant received prior to the start of the simulation.
- negativeRatings ($\text{negativeRatings} \geq 0$) The number of negative ratings the restaurant received prior to the start of the simulation.
- recipes (*array of recipe : id*) The recipes the restaurant owns.
- kitchenStaff (Kitchen Staff) The properties of the restaurant kitchen.
- waitstaff ($\text{waitstaff} > 0$) The number of waitstaff the restaurant employs.
- tables (*array of table*) The tables in the restaurant.

Figure 1: Restaurant properties that are specified in the JSON file

- 1 ingredients for the favorite dish for all customers in the event group. For the seats on all
- 2 other tables in the front of house, the cooks have to guess what the customers might be
- 3 eating. Thus, they estimate that they will receive orders for every dish on the menu at least

Kitchen Staff

- EXEC ($EXEC \in \{0, 1\}$) The number of head cooks.
- SOUS ($SOUS \geq 0$) The number of second-in-command cooks.
- TOURNANT ($TOURNANT \geq 0$) The number of floater cooks.
- SAUCE ($SAUCE \geq 0$) The number of sauce cooks.
- FISH ($FISH \geq 0$) The number of fish cooks.
- ROAST ($ROAST \geq 0$) The number of roast cooks.
- VEGETABLE ($VEGETABLE \geq 0$) The number of vegetable cooks.
- PASTRY ($PASTRY \geq 0$) The number of pastry cooks.

Figure 2: Kitchen Staff properties that are specified in the JSON file

1 $\lceil \frac{\#otherSeats}{10} \rceil$ times. The cooks sum up all required ingredients for those dishes and check
2 against the inventory of the pantry. The cooks buy all ingredients of which there isn't enough
3 available, including recipes for which they currently have no eligible cook. Those ingredients
4 are bought in the next bigger packaging volume from the supplier and immediately transferred
5 to the restaurant's pantry.

6 Different types of cooks work in the kitchen: the head cook (EXEC), their deputy (SOUS),
7 general cooks (TOURNANT) and several specialized cooks (SAUCE, FISH, ROAST, VEGETABLE,
8 PASTRY). The cooks are ranked in that order when deciding which cook will cook a meal,
9 including the ranking within the specialized cooks. Not every restaurant has every position
10 occupied. Except for the EXEC cook, all positions can be occupied more than once. The
11 cooks have different kinds of capabilities when it comes to making the recipes, so the recipes
12 contain information about which cook types can cook that dish (e.g., a roast should not
13 be cooked by the pastry cook) and how long it takes.

14 Once an order is placed by the customer, it is immediately registered in the kitchen, the order
15 is queued, and the ingredients are reserved in the exact amount required by the recipes. The
16 remaining portions of packages remain in the pantry for future orders. Orders queue waiting
17 for the next eligible cook to be available. If there are multiple eligible cooks for a dish within
18 an order, the dish is assigned to the lowest-ranking eligible cook. For cooking, the cooks can
19 profit from scaling effects: When the cook starts with their assigned dish, the cook doesn't
20 just cook the meals wanted in that single order but for all orders that include this dish. The

1 cook takes the reserved ingredients for all orders of that dish from the pantry and starts
2 cooking the meals, without an additional increase in duration. The cooking process will take
3 the duration given in the recipe, which will last $\lceil \frac{\text{duration}}{10} - 1 \rceil$ ticks. At the end of cooking,
4 the cook communicates to the waitstaff that the meals have been cooked. A cook cannot
5 cook another recipe in parallel, and they can only start working on another recipe in the
6 subsequent tick. However, it is possible that another cook might start cooking the same dish
7 in a following tick while this cook is still occupied. Cooks are seen as interchangeable, they
8 do not have ids assigned at the start of the simulation. To keep track of their actions, you
9 will assign them ids starting from $\text{id} = 1$ per restaurant and evening. A cook is assigned
10 an id the moment they start cooking the first assigned dish, and they will keep it for the
11 remainder of the evening.

12 In parallel to cooking, the cooks also maintain a list of which ingredients are available (i.e.,
13 in the pantry and not reserved for other meals) and thus, which dishes can still be ordered.
14 A dish can be ordered exactly if the required ingredients are available in the pantry and at
15 least one eligible cook exists in the restaurant.

16 It can happen that a meal in an order is aborted, e.g., when a customer has been waiting
17 for too long and leaves the restaurant, or a meal is not finished at the end of the opening
18 time. ~~All ingredients that have been reserved for such a meal or that have been used in an~~
19 ~~unfinished meal will be returned to the pantry.~~

20 All ingredients that have been reserved for such a meal or that have been used in an
21 unfinished meal will be used and eaten by the staff, as the restaurant cannot guarantee that
22 the storing conditions were kept.

23 Front of house (FOH)

24 The front of house (FOH) is the area containing all tables and seats, which are serviced
25 by the waitstaff. The waiters perform the actions of type SEATING the customers on ta-
26 bles, TAKING ORDERS, SERVING MEALS, receiving payment and ESCORTING them outside the
27 restaurant.

28 At the beginning of the evening, the waitstaff manager reserves one table for each EVENT
29 and then REGULAR group. This table will only be used by the group for which they were
30 reserved, even after the group has left the restaurant. The table reservation occurs according
31 to the following criteria:

- 32 1. All occupied or reserved tables are ignored, so that there is at most one group per
33 (merged) table.
- 34 2. In case the group size fits a table perfectly, they are assigned that table.
- 35 3. In case there are tables with a larger size available, if the group size is at least three
36 quarter of the table's size, they are assigned that table.

1 4. In case there are smaller tables, those smaller tables will be merged from the smallest
2 to the largest table until the group size is met. In case the final table size is above
3 the group size, the smallest table in this merge is removed iteratively (in order of
4 ascending id), so long as the table size remains greater or equal to the group size.
5 The group must cover at least three quarter of the resulting merged table.

6 As REGULAR and EVENT groups are of special importance to a restaurant, the manager always
7 tries to find a table for those groups if possible. For this reason, the manager lifts the three
8 quarter rule if no table has been found:

9 5. Perform step 3 without the three quarter limit, and if there is a result, assign that
10 table.

11 6. Perform step 4 without the three quarter limit, and if there is a result, assign that
12 table.

13 In general, if there are multiple tables that fulfill a criterion, the ones with the smallest size
14 and then lowest id are chosen first. Merged tables will subsequently operate on the lowest
15 id owned by the original tables. If a REGULAR or EVENT group cannot be assigned a table,
16 the manager informs them immediately, so they do not show up at the restaurant. This
17 might result in a negative rating.

18 During the serving phase, the waiters can perform actions every tick. The waiters are seen
19 as interchangeable, they do not have ids assigned at the start of the simulation. To keep
20 track of their actions, you will assign them ids starting from id = 1 per restaurant at the
21 moment they perform their first action each evening. Each waiter has a limit of 10 actions
22 per type and tick, so for example they cannot SEAT more than 10 people to a table in that
23 tick.

24 To evaluate the work load of the waitstaff, we use two characteristics: The *current load* is the
25 amount of customers a waiter is waiting on, so any customer this waiter has SEATED before
26 they are ESCORTED out of the restaurant. EVENTS do not count toward that load. The *tick*
27 load is defined per action type and describes how many actions of that type the waiter has
28 performed that tick.

29 The moment a customer group arrives (their visitingTick), a free waiter greets them and
30 SEATS them on a table among those of the required table type. ~~In case there is no free~~
31 ~~waiter, the customers try again the next tick and then leave, they do not reconsider.~~ In case
32 there is no free waiter, the customer group tries again the next tick and then leaves, the
33 group does not reconsider.

34 As the waitstaff manager wants to distribute customers well among the available waitstaff,
35 the manager assigns according to the following rules:

36 1. To assign a free waiter, the manager only considers those with a SEATING tick load
37 below the limit who won't exceed it with the new group's size.

38 2. The first candidate pool the manager selects from is all waiter with a current load
39 below 10, as they do not have much to do yet. Among those, the waiter with the

1 most customers is selected, so that the waiter gets a suitable workload and the other
2 waiters have the capacity for SEATING bigger groups still.

3 3. In case there is no waiter with current load below 10, we consider all waiters with the
4 sum of SEATING tick load and group size below the action limit. Here, the waiter
5 with the least current load is selected first, to balance the current load.

6 4. In case of a tie, the waiter with the lowest id is selected.

7 The assigned waiter then prepares to SEAT the customers to a table. For REGULAR and
8 EVENT groups, which are seated first within a tick, the table has been determined at the
9 beginning of the serving phase. For CASUAL groups, the table is created and assigned ad-hoc
10 immediately after assigning a waiter. For this, the rules 1–4 of the table assignment above
11 are applied consecutively. In case no table is free or eligible, the customers are sent away
12 and the waiter does not register this as a SEATING action. For both REGULAR and CASUAL
13 groups, the selected waiter is assigned to the group and table and will be responsible for all
14 further actions.

15 This waitstaff assignment counts for REGULAR and CASUAL groups. The EVENT groups are
16 greeted by the event manager personally and there exist no assigned waitstaff, as the manager
17 decides ad-hoc which waiter has to perform an action. Any waiter with a SEATING tick load
18 below the limit can be considered. To select the waiter, the manager prioritizes the waiters
19 in descending order of the current load to perform the SEATING, and each waiter SEATS
20 customers from the EVENT group until their SEATING tick load is at the limit. For example:
21 For a group of 15 customers, the first waiter, who has a current load of 15 customers but
22 SEATING tick load of 0 will seat 10 of the customers. The next waiter, who has a current
23 load of 11 customers and a SEATING tick load of 6 will seat the next 4, and a last waiter
24 with no current load or tick load will seat the last one. An EVENT SEATING only counts as
25 fulfilled if all customers in the group could be seated within the tick. Otherwise the attempt
26 counts as failed and the EVENT group will leave the restaurant. CASUAL groups arrive after
27 EVENT SEATING has been completed.

28 Immediately after SEATING the customers on the table, the customers order as a group
29 and the waiter who seated them also TAKE THEIR ORDERS. Orders are assigned an id that
30 increments starting from 1 and is unique across all restaurants and all evenings. The order
31 is taken on a table basis, where every customer chooses their preferred options based on the
32 available dishes. The customers directly talk to the waiter, who can check the availability of
33 dishes in their system, so the waiter informs every customer in the group which dishes are no
34 longer available to order. The waiter transfers the orders immediately to the kitchen.

35 Once the food is ready, the next action for the waitstaff is to SERVE the food to the customers.
36 A cooked meal is not immediately served, as the waitstaff prioritizes SERVING complete tables
37 within the same tick. Thus, after the first meal has been cooked, the table is not SERVED
38 for this and the following tick unless all other meals have been cooked as well. If not all
39 meals for a REGULAR or CASUAL table have been cooked in that time window, the assigned
40 waiter will start SERVING the meals to the table one by one as soon as they have been
41 cooked. If the assigned waiter does not have enough SERVING capacity to serve the full

1 table even though the meals have been cooked, the waiter will wait for the next tick and
2 then start SERVING the meals to the table one by one. They prioritize basic dishes, and in
3 a tie sort by ascending recipe id. ~~SERVING is first performed for REGULAR and then CASUAL~~
4 ~~groups.~~ SERVING is first performed for REGULAR, then EVENT and then CASUAL groups. For
5 EVENT groups, the priority for SERVING complete tables within the same tick still applies.
6 For each EVENT table, the manager checks if there are meals to be served in accordance with
7 that priority. If meals have been waiting too long in the kitchen or the table has already
8 been SERVED at least one meal, the manager simply assigns as many waiters as needed to
9 SERVE as many meals as possible to that table. Otherwise, the manager checks the SERVING
10 capacity of the whole waitstaff. If the sum of those SERVING capacities suffices to serve the
11 EVENT table, the manager will assign as many waiters as needed. Otherwise, the manager
12 will wait one tick and then start SERVING meals one by one in the next tick. In general, the
13 manager prioritizes the waiters in descending number of cooked meals in the kitchen that
14 belong to their assigned tables.

15 For deliveries, the meals queue until all meals of an order are ready and a driver is free.
16 Then, all waiters with a SERVING tick load below the action limit deliver meals to the drivers
17 waiting outside the restaurant until their limit is reached or the condition is negated. The
18 waiters with the lowest id starts, and the order with the lowest id takes precedence.

19 After the customers of a table have eaten, the waitstaff collects payment, ESCORTS the
20 customers outside, cleans and potentially restores the tables. ESCORTING can be performed
21 also partially in case the waitstaff doesn't have enough capacity left. Tables cannot be
22 changed until the group sitting on them has left completely. For EVENT groups, the waitstaff
23 manager prioritizes waiters with the lowest current load.

24 In a tie situation, the waitstaff first acts for the groups based on group type and then lowest
25 id.

26 Table

27 The front of house contains one or multiple tables with varying properties. Each table has a
28 unique id per restaurant. Tables have different types: They can be placed in the COMMON
29 room, describe spaces at the BAR, or be positioned inside a SEPARATED room. Each table
30 has a size between 2 and 30 seats to which individual people can be assigned. Tables of the
31 same type can be merged in case a bigger group needs more space, except for BAR tables.
32 Commonly, a table should be SERVED within one tick.

33 Ingredients

34 The restaurants obtain ingredients from the supplier and cook meals out of them. There
35 exist different kinds of ingredients, which are measured in one of the following units: g,
36 X, or mL. Each ingredient has one fixed packaging volume, which measures the amount
37 the supplier sells it in. The restaurants can only pick up ingredients in a multiple of their

Table

- id ($\text{id} \geq 0$) The unique identifier of the table.
- type ($\text{type} \in \{\text{COMMON}, \text{BAR}, \text{SEPARATED}\}$) The type of the table.
- size ($\text{size} \in [2; 30]$) The number of seats that are on the table.

Figure 3: Tables that are specified in the JSON file

- 1 packaging volume. All ingredients have a best-before date that restaurants have to
- 2 adhere to, to ensure high quality of the food and to not risk health issues for the customers.
- 3 The best-before date starts from the time the restaurant obtains the ingredient and is
- 4 measured in days, including the evening of obtainment.

Ingredient

- name (name) The unique name of the ingredient.
- unit ($\text{unit} \in \{\text{G}, \text{X}, \text{ML}\}$) The unit to measure the ingredient in.
- packagingVolume ($\text{packagingVolume} > 0$) The base amount in which the ingredient can be obtained from the supplier.
- bestBefore ($\text{bestBefore} > 0$) The number of days, including the day the ingredient will be obtained, that the ingredient will last.

Figure 4: Ingredients that are specified in the JSON file

5 Dishes

- 6 Restaurants offer dishes on their menu that customers can order. The cooked instances are
- 7 called meals, which are cooked in the kitchen, SERVED by the waitstaff and eaten by the
- 8 customer. Dishes are defined by a recipe and are cooked absolutely following the recipe. All
- 9 restaurants with the same restaurant type have the same basic dishes per default, however
- 10 they might vary in their recipes. In addition, restaurants may have additional dishes. The
- 11 basic dishes are defined in the recipe objects. There must be at least one basic dish and
- 12 recipe per restaurant type.

1 Dish names must be unique per restaurant. Meals are cooked ad-hoc when ordered, not
2 before. Meals are cooked for a specific customer group, they will not be served to another
3 group.

4 Recipes

5 Recipes describe the resources needed to cook meals based on ordered dishes. Each recipe
6 has a dish name, a duration, a set of ingredients along with their required amount,
7 and the cook type capable of following the recipe to cook the resulting meal. Recipe
8 ids are unique across restaurants. Dish names are unique per restaurant, but not across
9 restaurants. All restaurants with the same restaurant type offer the same basic dishes. As
10 restaurants can adapt the recipes for basic dishes, the ingredients, duration and cook
11 type may vary from the default recipe of that basic dish.

Recipe

- id ($id \geq 0$) The unique identifier of the recipe.
- dishName (dishName) The name of the dish.
- duration ($duration \in [2; 40]$) The duration of how many minutes it takes to cook a meal of a dish.
- cookType ($cookType \subseteq \{EXEC, SOUS, TOURNANT, SAUCE, FISH, ROAST, VEGETABLE, PASTRY\}$)
The type of staff that can cook the dish.
- ingredients (*array of*
(*name : ingredient : name, unit : ingredient : unit, amount : int > 0*))
The list of ingredient names with their unit and integer amount.
- basicDishFor ($basicDishFor \in \{AFRICAN, AMERICAN, ASIAN, EUROPEAN\}$)
The restaurant type for which this dish is a basic dish for and this recipe is the default recipe, optional.

Figure 5: Recipes that are specified in the JSON file

1 Customer Groups

2 Customer groups add a dynamic element to the simulation, as they have a lot of variation.
3 Groups have a size that defines how many customers there are in the group. Customer
4 groups can choose different table types to sit on. The groups can arrive at various times
5 within the evening (visitingTick). And the individual customers in the group can have dif-
6 ferent preferences for food. In addition, our simulation differentiates between different
7 types of customer groups.

8 The first type are REGULAR groups, which visit a specific restaurant every fixed number
9 of evenings. This fixed number of evenings is defined by visitingStart, which is the first
10 evening the group visits the restaurant, and visitingPeriod which determines how many
11 evenings later the group will visit again after their last visit. For example, visitingStart= 3
12 and visitingPeriod= 2 mean that the group will visit on evening 3 and then on 5, 7, 9,
13 11, and so on. Restaurants reserve tables for them in advance during the whole evening,
14 including after the group has left. Every regular group creates a rating for every visit to
15 promote their restaurant. After two consecutive failed attempts, which are either failed
16 reservations, failures to be seated by the waitstaff or the whole group leaving the restaurant
17 because no one was served food, they do not visit a restaurant again.

18 Another type are EVENTS. Customers can reserve space in restaurants for an event at a
19 fixed event evening. EVENT groups don't have a predetermined restaurant. Instead, they
20 only have certain restaurant types they consider, and they choose their restaurant via
21 the browsing and rating service. They reserve 3 evenings before the event occurs in the
22 first tick of that evening, to enable the restaurant to reserve tables and prepare ingredients.
23 For this reason, EVENT groups will visit restaurants starting from the fourth evening. Each
24 EVENT group has previously decided on a favorite basic dish which it recommends to
25 all members and for which the restaurant prepares. Similar to REGULAR groups, restaurants
26 reserve tables for EVENTS for the whole evening. EVENT groups always leave a rating.

27 After those have been settled, restaurants offer their remaining space to CASUAL customer
28 groups, such as tourists or irregularly visiting locals. They have defined evenings (the
29 visitingEvenings) at which they get food from restaurants, but are not bound to a
30 specific restaurant. Instead, they identify the restaurant each evening via the browsing and
31 rating service based on restaurant types they consider. CASUAL customers can request
32 deliveries, which is affected by the delivery distance between group and restaurant loca-
33 tion, or go to a restaurant in person. Those going to a restaurant decide on the restaurant
34 in the visitingTick by using the browsing and rating service. Those customers wanting
35 deliveries decide on a restaurant and make their order early. They factor in the delivery
36 distance and add 3 ticks for cooking. This way, an order that will be cooked immedi-
37 ately can always arrive at the visitingTick. Each casual customer group has a rating
38 likelihood that indicates how likely they are to give a rating about their restaurant expe-
39 rience. For a likelihood of NEVER, the group never creates a rating. With SOME, the group
40 will create a rating for a negative or positive experience. In case of ALWAYS, the group will
41 additionally make a positive rating in neutral experiences.

Customer group

- id ($id \geq 0$) The unique identifier of the customer group.
- type ($type \in \{\text{REGULAR}, \text{CASUAL}, \text{EVENT}\}$) The type of the customer group.
- size ($type \in \{\text{REGULAR}, \text{CASUAL}\} \Rightarrow size \in [1; 10] \text{ else } size \in [4; 100]$) The size of the customer group.
- tableType ($tableType \in \{\text{COMMON}, \text{BAR}, \text{SEPARATED}\}$) The type of table the group wants, in case it is not a delivery, which is optional with default to COMMON.
- visitingTick ($visitingTick \in [1; 21]$) The tick within an evening when the customer group wants to get food from a restaurant.
- foodPreferences (*array of foodPreferences*) Food preferences for customers within the group.
- visitingStart ($visitingStart > 0$) The first evening the group goes to the restaurant in the simulation, in case it is a regular group.
- visitingPeriod ($visitingPeriod \in [1; 10]$) The number of evenings at which a group goes to a restaurant periodically, in case it is a regular group.
- restaurant ($restaurant \geq 0$) The id of the restaurant a group goes to periodically, in case it is a regular group.
- restaurantTypes ($restaurantTypes \subseteq [\text{EUROPEAN}, \text{ASIAN}, \text{AFRICAN}, \text{AMERICAN}]$) The types of restaurant to visit in case it is a casual group or an event.
- eventEvening ($eventEvening \geq 4$) The evening a group intends to get food from a restaurant, in case it is an event.
- favoriteDishes (*favoriteDishes dict of restaurant type to favorite basic dish name*) The favorite dish to be selected from the basic dishes, in case it is an event.
- visitingEvenings (*array of int* > 0) The evenings a group intends to get food from restaurants, in case it is a casual group.
- deliveryDistance ($deliveryDistance \geq 0$) Distance between customer and restaurant in kilometers, in case it is a casual customer group. *deliveryDistance* > 0 means it is a delivery.
- ratingLikelihood ($ratingLikelihood \in \{\text{NEVER}, \text{SOME}, \text{ALWAYS}\}$) The likelihood of a group creating a rating after a restaurant experience in case it is a casual group.

Figure 6: Customer groups that are specified in the JSON file

- 1 **Food preferences** CASUAL customers and EVENTS choose their restaurant among available
- 2 options via the browsing system. For this, customers and restaurants offer multiple features
- 3 that must be considered. Customer groups decide on one or multiple restaurant types
- 4 that they want to visit, stated in the customer JSON. In addition, customers within a

1 group can name ingredients they prefer to eat, exclude from eating, as well as favorite
 2 dishes. Each preference must use 1–3 of those properties. The sum of all subgroup sizes
 3 must be less or equal to the customer group's size.

Food preferences

- size (size) The size of the subgroup in the customer group that adheres to those food preferences.
- excludedIngredients (excludedIngredients $\subset$ ingredients : name)
The ingredients the subgroup doesn't want to eat, optional.
- preferredIngredients (preferredIngredients $\subset$ ingredients : name)
The ingredients this subgroup would like to eat if given the choice, optional.
- favoriteDishes (favoriteDishes $\subset$ recipes : dishName) The dishes this subgroup favors above others, optional.

Figure 7: Food preferences that are specified in the JSON file

4 **Ordering** Once on their tables, or for deliveries immediately after having decided on a
 5 restaurant, the customer group immediately orders their dish. Each customer in the group
 6 decides on their dish based on the following rules: They only select among dishes where
 7 they eat all ingredients (no excluded ingredients). Among those, they choose their first
 8 matching favorite dish. If that is not possible, they select the dish that contains the most
 9 of their preferred ingredients, based on enumeration not quantity. In case there are
 10 multiple dishes left, they select the one with the highest recipe id. *Among dishes where they*
 11 *eat all ingredients, event customers prioritize the favorite dish of the event over their own*
 12 *favorite dish or preferred ingredient.* If there are no preferences, customers select
 13 the dish with the highest recipe id.

14 **After ordering** After ordering in a restaurant, customers expect food within 4 ticks but
 15 wait for up to 5 ticks for their food to be SERVED. If they haven't received food until then,
 16 the group leaves immediately. If at least one person on the table has received their meal,
 17 they wait for 2 more ticks. Then, if a customer on that table still doesn't have their meal,
 18 this customer leaves immediately. Customers with a delivery order wait until 3 ticks after
 19 the tick when they wanted it to arrive, afterward they abort their order. Once a customer
 20 has received their meal, they take 2 full ticks to eat it.

21

1 Rating

2 At the end of the tick where in-restaurant customer groups have left the restaurant or
3 customer groups using the delivery service have eaten their food (in case they didn't get
4 food at the end of the 3rd tick after the tick where their food should have arrived), or in
5 the first tick of the evening when the reservations failed, the customer groups can rate their
6 experience and leave a rating in the browsing and rating service. Some customer groups only
7 rate if their experience was especially good or bad, and others rate never or always.

8 An experience counts as negative in case there is no staff or table available and the customer
9 group has to leave the restaurant without getting seated. This includes if a regular or event
10 group planned for a restaurant but the table reservation was canceled that day. Additionally,
11 a negative experience is if the food arrives too late or not at all for at least one customer
12 in the group. For deliveries, it can also mean that they were aborted or were rejected. A
13 neutral experience is when all food for the whole group arrives in time, for deliveries at the
14 visitingTick. An experience counts as positive if the food for the whole group arrives
15 before the 4 tick expectation window is over. For deliveries, it is if the food arrived before
16 the visitingTick.

17 All customers that leave a rating will leave a positive rating in case of neutral experiences.
18 The exceptions are CASUAL customers with SOME, who will not rate after a neutral experi-
19 ence.

20 Browsing and Rating Service

21 Some customers use a service to find their restaurant. This service has global information
22 about all restaurants, which includes static information such as the restaurant type, opening
23 times, as well as whether they can deliver food or host events. The browsing system also
24 knows which dishes a restaurant has recipes for and what ingredients those dishes contain.
25 In addition, the service also knows for the current tick per table type how many seats are
26 available (neither on a reserved or occupied table), so that customers do not go to full
27 restaurants. They also store how many delivery drivers are currently available. In addition,
28 they store ratings made by customers, which also impact the choice of other customers.

29 To ensure that the customer groups do not flood a single restaurant with customers, the
30 service estimates the amount of customers planning to visit a restaurant or request a delivery
31 and uses this knowledge to update the availability of seats inside a restaurant. Once a cus-
32 tomer group has decided on a restaurant, the service reduces the number of seats according
33 to the group size, or decrements the number of delivery drivers available, so that the next
34 customer will be offered restaurants based on the updated availability estimation per table
35 type.

36 The time of using the service depends on the customer type and whether it is a delivery. For
37 using the service, a customer group enters their wanted restaurant types and table type.

1 **EVENTS** groups enter their **eventEvening**. The group receives only eligible matches, based
2 on:

- 3 ▪ for **CASUAL** groups whether the restaurant is currently open
- 4 ▪ for **EVENTS** groups whether the restaurant will be open at the **visitingTick** of the
5 **eventEvening**
- 6 ▪ whether a delivery driver is currently free in case of a delivery
- 7 ▪ for **CASUAL** groups whether for the required table types there are still available seats
8 based on prior reservations and the estimated number of customers already going to
9 the restaurant
- 10 ▪ for **EVENTS** whether there are still available seats based on the group sizes of prior
11 **EVENT** reservations

12 Based on this information, the group looks only at restaurants where every customer finds
13 at least one dish consisting only of ingredients they eat. Among those, they select the
14 restaurant with the highest difference between positive and negative ratings. In a
15 tie, they choose the restaurant with the lowest id. With this decision made, the customer
16 group approaches only the selected restaurant for the interaction, they do not reconsider.
17 For **EVENTS**, this means that after this decision has been made, the customer group makes a
18 reservation at the restaurant immediately. In case there is no restaurant that can be selected,
19 the attempt is aborted immediately.

20 **Delivery Service**

21 After deciding on the restaurant, some customer groups contact the attached delivery service
22 and make their food choices. They then order their dishes as described in the ordering part,
23 the order is subsequently delivered to the restaurant. The drivers are waiting outside the
24 restaurant, so they can start delivering immediately after being given the ordered meals
25 by the waitstaff. The delivery will start in the tick after finishing the meals, and will last
26 $\lceil distance/5 \rceil$ ticks. At the end of this last tick, the customers receive their meals. This
27 time might be after the restaurant has already closed. The driver needs the same amount
28 of ticks to return to the restaurant, before they can deliver a new order. Each driver will
29 carry out only one customer group's orders. ~~Deliveries where it is clear at the start that~~
30 ~~they will not be cooked before the end of the opening time are rejected (cooking time~~
31 ~~would be over after restaurant is closed).~~ Drivers are seen as interchangeable, they do not
32 have ids assigned at the start of the simulation. To keep track of their actions, you will
33 assign them ids starting from $id = 1$ per restaurant at the moment they receive meals each
34 evening.

1 End of the opening time / evening

2 A restaurant does not accept new customers in the last 3 ticks of their opening time.

3 If a restaurant has reached the end of the opening time and customers are still inside,
4 you can assume that they are all escorted outside immediately. Those groups that have
5 finished eating see this as a non-negative experience, for all other groups this counts as a
6 negative experience. The kitchen stops working, unused ingredients remain in the pantry and
7 reservation from this evening are discarded. All started or cooked meals are thrown away or
8 eaten by the staff. The front of house is immediately cleaned and tables are separated in
9 preparation of the next evening. Only deliveries already given to a driver continue after the
10 opening time of the restaurant until the end of the evening.

11 The end of the evening is almost identical to the end of the opening time, except here also
12 deliveries are aborted. Deliveries on their way to a customer are aborted, without rating or
13 other consequences. In contrast, deliveries where the driver is on the way to the restaurant
14 continue. Drivers immediately go home and return the next evening.



15

Hint for implementation

It is clear that the described operations do not perform optimally. This is accepted to deliver both a deterministic and rather simple, greedy algorithm (compared to solving a planning problem and optimizing the restaurant throughput).



16

Hint for implementation

If not otherwise specified, prefer the lowest id as final tie-break. For ingredient or dish names, use the alphabetic order.

17 Incidents

18 In our simulation, we have different incidents that occur over the course of the day and
19 shape the evening's status. Each incident has a unique id, an incident type, an evening
20 when the incident occurs, and a set of properties that define the impact of the incident on
21 the simulation.

1 **Staff Change**

2 During the course of the simulation, it can occur that the staff in a restaurant changes,
3 that members quit or new members are recruited. The incident has a non-zero number
4 property that indicates how many staff members will be added (positive number) or removed
5 (negative number). It also contains a restaurant property that contains the id of an
6 existing restaurant in the simulation. In addition, the incident contains multiple properties
7 to describe the type of staff that is affected, of which only one should be used: `staffType`
8 sets the type of staff (COOK, WAITSTAFF, DRIVER) that the change affects. In case it is COOK,
9 the `cookType` property contains the type of cook that shall be changed. This can be any
10 cook type but EXEC. All staff numbers remain non-negative, so a reduction beyond 0 results
11 in having 0 members of that staff type.

12 **Recipe Change**

13 Restaurants are impacted by the availability of ingredients and the political regulations. Thus,
14 restaurants adapt their recipes globally to face those challenges. This incident changes
15 all recipes in all restaurants that use a specific ingredient, modifying the presence of the
16 ingredient via a relative factor. The ingredient to be changed is captured with the property
17 `ingredient`. The incident also has a non-zero `adaptation` property that specifies by
18 what percentage the amount of an ingredient increases (+) or decreases (-) within a recipe
19 compared to its prior amount. The resulting value is always rounded down to a full integer but
20 must remain positive. For example, adaptation of 3 results in a 3% increase, transforming
21 50 g ONIONS into 51 g.

22 **Packaging Change**

23 Food manufacturers, driven by their greed and adapting to the local economy situation, may
24 change the packaging volume of ingredients. This incident changes the packaging volume
25 of one ingredient to another size. The ingredient for which the change occurs is captured
26 with the property `ingredient`. `packagingVolume` contains the new size of the packaging
27 volume, which must be positive.

28 **Ingredient Unavailability**

29 Due to problems in the supply chain, some ingredients may become temporarily unavailable
30 for the supplier. This incident renders an ingredient unavailable for a fixed amount of
31 evenings. The ingredient to be changed is captured with the property `ingredient`. The
32 incident also has a positive `duration` property that states for how many evenings, including
33 the one in which the incident occurs, the ingredient will remain unavailable. During those
34 evenings, the restaurant will not be able to purchase the ingredient from the supplier. For a
35 given ingredient, there must not be any overlaps in the duration.

Incident

- id ($\text{id} \geq 0$) The unique identifier of the incident.
- type ($\text{type} \in \{\text{STAFF}, \text{RECIPE}, \text{PACKAGING}, \text{UNAVAILABLE}\}$) The type of the incident.
- evening ($\text{evening} \geq 0$) The evening when the incident occurs.
- restaurant ($\text{restaurant} \geq 0$) The restaurant in case of staff change.
- number ($\text{number} \neq 0$) The number of staff members to change in case of staff change.
- staffType ($\text{staffType} \in \{\text{COOK}, \text{WAITSTAFF}, \text{DRIVER}\}$) The type of staff for which the change occurs in case of staff change.
- cookType ($\text{cookType} \in \{\text{SOUS}, \text{TOURNANT}, \text{SAUCE}, \text{FISH}, \text{ROAST}, \text{VEGETABLE}, \text{PASTRY}\}$) For which type of cooks a staff change occurs in case of staff change with type COOK.
- ingredient ($\text{ingredient} : \text{ingredient} : \text{name}$) The name of the ingredient in case of recipe change, packaging change or ingredient unavailability.
- adaptation ($\text{adaptation} \neq 0$) The adaptation in percent in case of recipe change.
- packagingVolume ($\text{packagingVolume} > 0$) The new packaging volume in case of packaging change.
- duration ($\text{duration} > 0$) The amount of evenings in case of ingredient unavailability.

Figure 8: Incident properties that are specified in the JSON file

1 2.3. Simulation Details

- 2 In this section, we will describe the overall behavior of the simulation, and the behavior of
- 3 the individual restaurants during the simulation. We also describe what information you need
- 4 to log so that we can assess your implementation from the outside.

1 2.3.1. Logging

2 As there is a lot of information that the simulation needs to store and keep track of, we use
3 log levels to allow specific logging of information. Commonly used to specify different levels
4 of severity, we use the log levels here to log information at different levels of detail. For this,
5 we will use the following three log levels:

- 6 ▪ **DEBUG**: With this log level, all of the logs specified in this chapter will be output.
- 7 ▪ **INFO**: With this log level, we remove all logs that do not report activities of the
8 simulation.
- 9 ▪ **IMPORTANT**: With this log level, we only include the most pertinent activities of the
10 simulation.

11 This log level will be specified in the command line via the argument `logLevel`.

12 In the following log descriptions, every log statement will have a log level assigned. It is
13 crucial to remember that **INFO** logs will also be output if a log level of **DEBUG** is specified,
14 and that **IMPORTANT** logs will be output no matter the log level.

15 Log statements usually contain variable parts, indicated by \$, where you have to fill in the
16 required information. Often, this is just one piece of information you can just state, but
17 occasionally, there might be multiple pieces of information you must describe in one variable
18 part (e.g., multiple table ids). In this case, the information must be sorted in ascending
19 order and output separated by commas (e.g., 3,5,7,8). In case key-value mapping must be
20 logged, output a comma-separated list of "key:value" pairs, (e.g., A:3,B:5,C:7,D:8).

21 2.3.2. Initialization

22 Before the simulation starts, the different configuration files have to be parsed and validated
23 to initialize the simulation. There are three different configuration files:

- 24 ▪ **Food** defines ingredients and recipes.
- 25 ▪ **Restaurant** defines the restaurants.
- 26 ▪ **Scenario** defines the customer groups and incidents.

27 Additionally, the user has to provide the maximum number of ticks the simulation should
28 run for as a command line argument (cf. Section 3.1). The schemes for the JSON files can
29 be found inside the repositories in the folder `src/main/resources/schema`.

30 **Configuration Parsing** The three configuration files will be provided as JSON files. You
31 can find more details on the structure of the JSON files in Section 2.2. After parsing the
32 JSON files, your program has to validate the configuration files according to the rules defined
33 in Section 2.2. Each configuration file has to be validated separately, and the simulation

1 should not start if any of the configuration files is invalid. Some constraints can be checked
2 in isolation for the different configuration files, but some constraints can only be checked in
3 the context of the other configuration files. Each constraint should be checked as early as
4 possible and the result renders the file valid or invalid.

5 The order of parsing and validation is as follows:

- 6 1. Parse and validate the Food configuration.
- 7 2. Parse and validate the Restaurant configuration.
- 8 3. Parse and validate the Scenario configuration.

9 After successfully parsing and validating each configuration file, your program has to out-
10 put to the output handle (which is provided as command line argument when starting the
11 simulation) a log message including the name of the configuration file.

```
12 [INFO] Initialization Info: $filename successfully parsed and  
validated.
```

13 In case the configuration file is invalid, your program has to output to the output handle a
14 log message containing the type of the log message (i.e., Initialization Info) and the
15 name of the configuration file.

```
16 [IMPORTANT] Initialization Info: $filename is invalid.
```

17 Validation

18 During the parsing, you have to validate the following constraints:

- 19 ■ All name and id properties that are specified as unique must be unique.
- 20 ■ The existence of objects which are referred to in other objects by name or id. For
21 example, ingredients used in a recipe should exist as defined objects.
- 22 ■ There must exist at least one ingredient and at least one restaurant.
- 23 ■ For every restaurant, openingTickStart must be less than openingTickEnd.
- 24 ■ Per restaurant there must exist at least one recipe, waiter, table and cook.
- 25 ■ There must exist at least one basic dish per restaurant type for which there exists a
26 restaurant.
- 27 ■ There must exist exactly 1 recipe per basic dish name.
- 28 ■ Each restaurant may have only 1 recipe per dish name.
- 29 ■ That the size of the customer group is greater or equal to the sum of size property
30 for the foodPreferences of this customer group.

- 1 ▪ For every object in `foodPreferences`, there can be no intersection between `excludedIngredients`
2 and `preferredIngredients`.
- 3 ▪ For `excludedIngredients`, `preferredIngredients`, and `favoriteDishes`, the
4 value array must be unequal to the set of all possible ingredients respective dishes.
- 5 ▪ Deliveries must have a `visitingTick` such that they make their order within the
6 serving phase (i.e., not before 6 pm). This must factor in the `deliveryDistance`.
- 7 ▪ Event customer groups need a favorite dish for every restaurant type they consider
8 visiting.
- 9 ▪ The duration of ingredient unavailable incidents must not overlap.

10 **Simulation Initialization**

11 After successfully parsing and validating the configuration files, the simulation can be initial-
12 ized. To initialize the simulation, you have to create the ingredients and recipes, restaurants
13 with their kitchen, front of house, and pantry, and the customer scenario, and global services
14 based on the information provided in the configuration files.

15 **Simulation Start** After the initialization of the simulation, the simulation starts. The
16 simulation is divided into evenings, and each evening is represented by a preparation phase
17 and a tick-based serving phase of 24 ticks.

18 To start the simulation, your program has to output to the output handle a log message
19 containing the type of the log message (i.e., `Simulation Info`).

```
20   [INFO] Simulation Info: Simulation started.
```

21 **Incidents**

22 First, the simulation logs all incidents that occurred before this evening in ascending order of
23 incident id, using the id and type of the incident as well as the number of evening.

```
24   [IMPORTANT] Incident: Incident $incident:id of type $incident:type  
      occurred before evening $evening.
```

25 **Preparation Phase**

26 Each evening starts with a preparation phase and outputs the following message to the output
27 handle.

```
28   [IMPORTANT] Preparation: Preparation for evening $evening starts.
```

1 Each restaurant passes the preparation phase as one in ascending id of the restaurant.
2 First, the waitstaff manager reserves tables for REGULAR and EVENT groups. For each group,
3 the manager reserves a specific set of tables to be merged upon SEATING. In case it was not
4 possible to reserve tables, the simulation logs the failure.

```
5 [IMPORTANT] FOH No Reserving (R $restaurant:id): No table could be reserved for group $group:id.
```

6 Based on those reservations, the kitchen starts planning. The kitchen removes expired ingredients from the pantry, computes the required ingredients and then orders the ingredients from the supplier. The simulation has to log a message containing the id of the restaurant and the amount, unit and name of each ingredient that was removed from the pantry. The order is by ascending alphabetic order of the name of the ingredient.

```
11 [DEBUG] Pantry (R $restaurant:id): Removed $ingredient:amount $ingredient:unit of $ingredient:name from the pantry.
```

12 The restaurant calculates the required ingredients and then procures them from the supplier.
13 The simulation has to output to the output handle the following log message containing
14 the id of the restaurant and the amount, unit and name of each ingredient that was
15 procured from the supplier. The order is by ascending alphabetic order of the name of the
16 ingredient.

```
17 [DEBUG] Pantry (R $restaurant:id): Procured $ingredient:amount $ingredient:unit of $ingredient:name from the supplier.
```

18 Finally, the restaurant signals that it has restocked the ingredients and finished the preparation phase.

```
20 [INFO] Pantry (R $restaurant:id): Restocked ingredients.
```

21 **Serving Phase**

22 The next phase of the evening is the serving phase. First, the simulation logs the start of
23 the serving phase for the evening.

```
24 [IMPORTANT] Serving: Serving of evening $evening starts.
```

25 The serving phase is performed in a tick-based manner from tick 1 to 24, with a consecutive
26 and deterministic order of actions within a tick. At the beginning of each tick, the simulation
27 has to output to the output handle that the current tick starts and which tick this is within
28 the evening.

```
29 [IMPORTANT] Simulation: Tick $tick ($tick:inEvening) started.
```

1 **Restaurant Decision** Each tick starts with the customer groups deciding on their restau-
2 rant in order of **group type (EVENT, CASUAL) and then** ascending group id. If it has not
3 been predetermined, a customer group decides on a restaurant, and the simulation logs this
4 step based on the id of the customer group and restaurant. **This step will also occur if the**
5 **customer group does not arrive at the restaurant this tick, e.g., for events.**

```
6 [DEBUG] Restaurant Decision: Group $group:id decided on restaurant  
$restaurant:id.
```

7 In case the group could not find a suitable restaurant, the simulation logs this using the
8 group id.

```
9 [DEBUG] Restaurant No Decision: Group $group:id could not decide for a  
restaurant.
```

10 The subsequent steps are restaurant-dependent and the logs will be performed separately
11 per restaurant in order of ascending id. First, the simulation logs the start of the restaurant
12 for this tick based on the restaurant id.

```
13 [DEBUG] Restaurant Start (R $restaurant:id): Restaurant $restaurant:id  
simulates a tick.
```

14 The remainder of the tick is processed in the following order for the restaurant:

- 15 1. Customer arrivals, seating and ordering
- 16 2. Kitchen scheduling and cooking
- 17 3. Serving
- 18 4. Delivering
- 19 5. Eating
- 20 6. Escorting customers
- 21 7. Rating

22 **Customer arrivals, seating and ordering**

23 This step is performed per customer group in multiple substeps. Each customer group
24 will only perform the steps they need to do. The order is per customer group type
25 (REGULAR, **EVENT**, CASUAL) and then ascending id.

26 **The following steps and logs are only required if the customer group is scheduled to visit a**
27 **restaurant this tick.** First, the group arrives at the restaurant. This is logged based on the
28 id of the customer group and restaurant.

```
29 [INFO] Restaurant Arrival (R $restaurant:id): Group $group:id arrived  
at restaurant $restaurant:id.
```

1 Then, this customer group is being assigned a waiter and SEATED. This is logged based on
2 the id of the group, the table and the waiter (one or more).

3 `[IMPORTANT] FOH Seating (R $restaurant:id): Group $group:id seated at
table $table:id by waitstaff $waitstaff:ids.`

4 In case tables had to be merged the simulation logs that before outputting the SEATING log,
5 based on the group id and the old and the merged table ids.

6 `[INFO] FOH Merging Tables (R $restaurant:id): For group $group:id the
tables $table:oldIds were merged into $table:mergedId.`

7 If there is no free waiter, the simulation logs that instead with the id of the group.

8 `[INFO] FOH No Seating (R $restaurant:id): No free waitstaff available
for group $group:id.`

9 If there is a free waiter but no seat available, the simulation logs that with the id of the
10 group and the waiter.

11 `[INFO] FOH No Seating (R $restaurant:id): Assigned waitstaff
$waitstaff:id but no table available, group $group:id is sent away.`

12 Customer group now decides on their dish and the waiter TAKES THEIR ORDER. This step
13 is performed for both restaurant visits and deliveries. Within a customer group, customers
14 order one by one, starting with the customer(s) that have the most excluded ingredients
15 and then the least favorite dishes. In a tie, the food preference object listed first in
16 the group's json has precedence. After each order has been placed the kitchen is informed
17 and the pantry updated. The orders are logged per group based on the id of the group, the
18 waiter (one or more) and the order, as well as a key-value mapping of dish name to amount.
19 In case the order is supposed to be delivered, the part about the waitstaff is left out, ending
20 the log after the key-value mapping with a "".

21 `[IMPORTANT] FOH Ordering (R $restaurant:id): Group $group:id placed
order $order:id of $dish:NameToAmount with waitstaff $waitstaff:id.`

22 In case one or more customers couldn't find a dish, *those customers* do not place an order
23 and leave the restaurant. This is logged per group based on the group id and the number
24 of customers.

25 `[IMPORTANT] FOH No Ordering (R $restaurant:id): Group $group:id could
not place an order for $customer:number customers, they leave the
restaurant.`

26 As a summary, the simulation logs the SEATING and TAKING ORDERS done this tick. First
27 is the seating based on the number of waitstaff performing seating and the number of
28 customers being seated and the number of (merged) tables on which they were seated.
29 Status logs like this one only log successful attempts, so failing attempts for SEATING or

1 taking an ORDER from a customer don't count in either the waitstaff, customer or table
2 number.

```
3 [DEBUG] FOH Seating Status (R $restaurant:id): $waitstaff:number  
waitstaff seated $customer:number customers on $table:number tables.
```

4 Next, the simulation logs the ordering status based on the number of customers ordering
5 a meal, either in the restaurant or for a delivery, and the number of waitstaff taking or-
6 ders.

```
7 [DEBUG] FOH Ordering Status (R $restaurant:id): The restaurant received  
orders from $customer:number customers, $waitstaff:number waitstaff  
took orders.
```

8 Kitchen scheduling and cooking

9 Next, the kitchen in each restaurant starts working. This step occurs now per order, and,
10 when more precision is required, per dish within each order. Per order in the queue, the
11 dishes of the order are assigned to a cook. Basic dishes take precedence, then the lower id
12 of the dish recipe. This is logged using the id and type of the cook and the id of the order
13 responsible for the cooking, the dish name, the number of meals that will be cooked and
14 the ids of all the orders that the meal will be cooked for.

```
15 [IMPORTANT] Kitchen Dish Assignment (R $restaurant:id): Cook $cook:id  
of type $cook:type starts cooking $number meals of dish $dish:name  
based on order $order:id for orders $allOrder:id.
```

16 Then, the kitchen cooks the meals for the duration of the tick and logs all meals that were
17 finished this tick. The order is by cook id. This step can include meals started both in this
18 and in prior ticks, i.e., a duration of 0 tick signifies that the meal is finished in the same tick.
19 The log contains the id of the cook, the number of meals, the dish name and the duration
20 in ticks since the waiter TOOK THE ORDER responsible for the cooking of this dish.

```
21 [IMPORTANT] Kitchen Meal Cooked (R $restaurant:id): Cook $cook:id  
finished cooking $meal:number meals of dish $dish:name $tick:number  
ticks after ordering.
```

22 As a summary, the kitchen logs the activity of all cooks in this tick. This includes the number
23 of cooks involved in cooking, the total number of meals they were cooking this tick (including
24 the finished ones), the number of meals finished cooking this tick and the number of meals
25 waiting to be served.

```
26 [DEBUG] Kitchen Status (R $restaurant:id): $cook:number cooks were  
active cooking $meal:totalNumber and finishing $meal:finishedNumber  
meals. $meal:servableNumber meals can be served by the waitstaff.
```

1 Serving

2 The next step is performed again by the waitstaff. They start SERVING meals according to
3 the rules provided in prior descriptions. The SERVING is prioritized by customer group type
4 (REGULAR, EVENT, CASUAL) and then ascending id. In case a waiter should SERVE a table
5 only partially, they prioritize meals of basic dishes and then ascending order of the id of the
6 dish recipe. The simulation logs this activity using the id of the waiter and table, a key-value
7 mapping of the meal's dish names to amount, and the duration since ordering in ticks (if
8 the order was placed this tick, the number to be logged is 0).

```
[IMPORTANT] FOH Serving (R $restaurant:id): Waitstaff $waitstaff:id  
serves $meal:dishNameToAmount to table $table:id $tick:number ticks  
after ordering.
```

9
10 In case there are meals the waitstaff could have served but did not due to waiting for other
11 meals of the table or reaching the action limit, the simulation logs these. For tables that
12 serve event customer groups, they log the last waiter that did serve or the first that would
13 have served. The log includes the id of the waiter and the table and the number of meals
14 left unserved.

```
[DEBUG] FOH No Serving (R $restaurant:id): Waitstaff $waitstaff:id did  
not serve $meal:number meals to table $table:id.
```

15
16 In case a waiter delivers meals to the driver, they log this as well using the id of the waiter,
17 driver and the customer order as well as a key-value mapping of the meal's dish names
18 to amount. The priority is by ascending order id, basic dishes vs other dishes and then
19 ascending dish recipe id.

```
[IMPORTANT] FOH Delivery (R $restaurant:id): Waitstaff $waitstaff:id  
serves $meal:dishNameToAmount meals to driver $driver:id for order  
$order:id.
```

20
21 As a summary, the waitstaff log all SERVING actions done this tick. This includes the
22 number of waitstaff performing SERVING and the total number of meals served or delivered
23 to a driver.

```
[DEBUG] FOH Serving Status (R $restaurant:id): $waitstaff:number  
waitstaff served $meal:totalNumber meals.
```

25 Delivering

26 The following logs describe the delivery process. The logs will be presented in sequential
27 order from the perspective of the driver, but not all will be performed for every driver / order
28 in a single tick. Each of the subsequent logs will be logged ~~per driver if they are in the state~~
29 ~~requiring that log~~ for all drivers that require that log, in order of ascending group id, and
30 then the next log will be output.

1 Based on the meals received by the waitstaff, the drivers that have a full order prepare
2 driving, which will start in the next tick. For that, they search the route and estimate the
3 number of ticks. The simulation will log this based on id of driver, customer group and
4 order as well as the estimated number of ticks.

5 [INFO] Delivery Preparation (R \$restaurant:id): Driver \$driver:id
prepares driving order \$order:id to group \$group:id, which will take
\$delivery:ticks ticks.

6 In the subsequent ticks, the driver will log the distance driven and the ticks remaining, which
7 can also be 0.

8 [DEBUG] Delivery Driving (R \$restaurant:id): Driver \$driver:id drove
\$delivery:distance km and needs \$delivery:ticks more ticks.

9 In case a driver arrived at the customer group, the driver will log that they arrived using
10 driver, group and order id.

11 [INFO] Delivery Arrival (R \$restaurant:id): Driver \$driver:id arrived
at group \$group:id with order \$order:id.

12 The driver will then, in the same tick, try to give the customers the ordered meals. In case
13 the customers accept the food and start eating, the simulation will log this using driver,
14 group and order id.

15 [IMPORTANT] Delivery Finished (R \$restaurant:id): Driver \$driver:id
gave delivery of order \$order:id to group \$group:id.

16 Otherwise, the simulation logs the failure of the delivery.

17 [IMPORTANT] Delivery Failed (R \$restaurant:id): Driver \$driver:id
failed to deliver order \$order:id to group \$group:id.

18 Customers can decide to give up waiting for a delivery in case the delivery was expected until
19 this tick but did not arrive until then. This is logged using group and order id in ascending
20 order of group id. All following delivery attempts of this order will fail.

21 [INFO] Delivery Given Up (R \$restaurant:id): Group \$group:id gave up on
waiting for delivery of order \$order:id.

22 After the delivery has been attempted, the drivers return to the restaurant driving the same
23 distance/ticks as on the way to the customer group. Once they have returned, the simulation
24 logs this based on driver and restaurant id.

25 [INFO] Delivery Returned (R \$restaurant:id): Driver \$driver:id has
returned.

1 Eating

2 Customers who weren't served for too long now leave the restaurant. This is logged in
3 ascending order of the customer group type and then group id and includes the id of the
4 table, the customer group and the number of customers leaving the table.

```
[INFO] Restaurant No Eating (R $restaurant:id): $customer:number  
customers of group $group:id leave table $table:id due to not being  
5 served.
```

6 Customers start eating immediately after being served. Once they have finished eating, this
7 will be logged. First, it is logged for every group sitting in the restaurant in order of customer
8 group type and then group id.

```
[INFO] FOH Finished Eating (R $restaurant:id): $customer:number  
9 customers of group $group:id have finished eating at table $table:id.
```

10 In case of deliveries, the simulation will also log for all groups being delivered to the following
11 log, in ascending group id.

```
[INFO] Delivery Finished Eating (R $restaurant:id): Group $group:id has  
12 finished eating.
```

13 As a summary, it will be logged how many customers are still eating or have finished eating
14 this tick within the restaurant.

```
[DEBUG] FOH Eating Status (R $restaurant:id): $customer:eatingNumber  
customers are eating and $customer:eatenNumber customers have finished  
15 eating this tick.
```

16 Escorting customers

17 Once all customers sitting on a table have finished eating, the waitstaff can escort them
18 outside the restaurant. The order is in ascending customer type and then group id, the
19 customers within the group do not matter. This is logged using the id of the waiter, group
20 and table as well as the number of customers the waitstaff is escorting

```
[IMPORTANT] FOH Escorting (R $restaurant:id): Waitstaff $waitstaff:id  
escorts $customer:number customers of group $group:id from table  
21 $table:id outside.
```

22 As a summary, it will be logged how many waitstaff were escorting and customers were
23 escorted this tick.

```
[DEBUG] FOH Escorting Status (R $restaurant:id): $waitstaff:number  
waitstaff escorted $customer:escortingNumber customers this tick.  
24
```

25

1 Ratings

2 Every group that has completely left the restaurant or that finished a delivery (finished eating
3 or giving up waiting on the delivery) in this tick will decide on whether they leave a rating
4 for the restaurant with the rating service. This step is done per group in ascending order
5 of the customer type and then group id, and the decision or rating will be logged based
6 on group and restaurant id. It is also logged how many positive and negative ratings the
7 restaurant currently has.

```
8 [INFO] Rating (R $restaurant:id): Group $group:id rates  
the restaurant $restaurant:id with $group:rating rating,  
leading to $restaurant:positiveRatings positive ratings and  
$restaurant:negativeRatings negative ratings.
```

9 As a summary, it will be logged how many groups rated this tick.

```
10 [DEBUG] Rating Status (R $restaurant:id): $group:number groups  
performed ratings this tick.
```

11 After all steps have been performed for a restaurant, the simulation logs that the restaurant
12 has finished simulating the tick.

```
13 [DEBUG] Restaurant End (R $restaurant:id): Restaurant $restaurant:id  
finished simulating the tick.
```

14 At the end of the serving phase, the simulation logs that the evening has ended.

```
15 [IMPORTANT] Serving: Serving of evening $evening ends.
```

16 If the maximum number of ticks is not a multiple of 24, the simulation ends in the middle
17 of the serving phase. In that case, the simulation continues after the Restaurant End
18 immediately with the Simulation Info. In case the maximum number of ticks is a multiple
19 of 24, the simulation ends with the serving phase, the log continues after the Serving
20 log with the Simulation Info. It will not proceed with the next evening's preparation
21 phase.

22 2.3.3. Simulation Statistics

23 After the simulation has ended, your program has to output to the output handle a log
24 message stating that the simulation statistics are calculated.

```
25 [IMPORTANT] Simulation Info: Simulation statistics are calculated.
```

26 To calculate the simulation statistics, you have to calculate the number of meals and the
27 number of customers the restaurants served or delivered, [as well as the ratings received](#).
28 Served and delivered meals count iff customers received their food. The statistics are logged
29 per restaurant based on its id. Afterwards you have to output the following log messages

1 for each restaurant (starting from the lowest id to the highest) containing the id of the
2 restaurant, ~~and~~ the amount of customers (and not visits) served and delivered to ~~and the~~
3 ~~amount of ratings received~~.

4 [IMPORTANT] Simulation Statistics: Restaurant \$restaurant:id cooked
\$meal:cookedNumber meals.

5 [IMPORTANT] Simulation Statistics: Restaurant \$restaurant:id served
\$customer:servedNumber customers.

6 [IMPORTANT] Simulation Statistics: Restaurant \$restaurant:id delivered
meals to \$customer:deliveredNumber customers.

7 [IMPORTANT] Simulation Statistics: Restaurant \$restaurant:id received
\$rating:number ratings.

3. Technical Details

3.1. Command-Line Interface

The simulator is started with these command line parameters:

--food Path to the file with information about the food. (always required) String
--restaurants Path to the file with information about the restaurants. (always required) String
--scenario Path to the scenario file. (always required) String
--maxTicks Maximum allowed number of simulation ticks. (always required, between 0 and 1'000) Int
--logLevel The level of log detail that shall be output (always required, either DEBUG, INFO or IMPORTANT) String
--out Path to the output file. Uses 'stdout' by default. String
--help Usage info

3.2. Build Script

We provide you with a build script. A build script is responsible for managing the dependencies of a software project and creating a finished executable program from the project files. As a build tool, we use *Gradle*¹.

The build script can be found in the `build.gradle.kts` file. Changes to your build script will be rolled back from the test server for execution. You can (and should) build the project yourself by either running the `./gradlew build` command or running the *Gradle* task build directly in an IDE (e.g., *IntelliJ*). We use *Kotlin 2.1.21* with the *Java JDK Version 21* from *Eclipse Temurin 21.0.8*. You can download it either via the website² or directly in *IntelliJ*. In case you download it via *IntelliJ*, please make sure to set `JAVA_HOME` to the folder of the Java installation and add its `bin` folder to the beginning of the `PATH` variable.

¹<https://gradle.org/>

²<https://adoptium.net/de/temurin/releases?version=21&os=any&arch=any>

1 In the build script there are dependencies on various libraries. We recommend that you look
2 at these libraries and consider whether you can use them. Other libraries that are not entered
3 in the build script must not be used. For security reasons, we do not allow reflection, network
4 connections, or other connections to the outside world. Also, with the exception of loading
5 configuration files and creating an output file, you may not read, create, or modify files in the
6 file system. For loading configuration files and writing an output file, you may only access
7 the paths given in the command line parameters and the *resources* folder for reading the
8 JSON schema.

9 **3.3. Code Quality**

10 Good code quality helps software to remain readable and it helps to avoid errors. In the SE
11 Lab, you will learn about automated tools that help you write good and (hopefully) bug-free
12 code. Specifically, we use the tool DETEKT³. You can use this tool with *Gradle*.

13 DETEKT is a static analysis tool, which means it doesn't need to run your code to find
14 errors. DETEKT examines the source code for certain patterns that are known to cause
15 errors. Examples of such patterns are unreachable code (you probably forgot to call a
16 method) or using 200 parameters in a function. You will notice that DETEKT does not allow
17 `print|println`. For the logging functionality, you have to use a `PrintWriter` from the
18 package `java.io`.

19 Furthermore, DETEKT provides style rules in order to check your code for style issues (e.g.,
20 missing documentation for public functions). This helps improve readability and maintain-
21 ability of your code.

22 You can find a report of the analysis in the directory `build/reports/detekt`.

³<https://detekt.dev/docs/intro>

4. Tests

In the group phase we expect system, unit, and integration tests from you. You can find the requirements, which are demanded from your application, in the specifications from the previous chapters.

Try to achieve the highest possible statement and branch coverage in the implementation phase. This means that in your code as many statements and branches as possible should be covered by the tests. The coverage can be calculated, e.g., with *JaCoCo*¹, also *IntelliJ* has already integrated tools for this.

4.1. Unit Tests

For unit tests, use *JUnit* 5² and *Mockito*³ which is integrated into the project via the *Mockito-Kotlin* wrapper. Various tutorials for using these frameworks can be found on the Internet⁴. We will provide you with a sample test at the beginning of the implementation phase.

The unit tests can be run with `./gradlew test` (or `gradlew.bat test`).



Hint for implementation

Unit tests belong in the given directory `src/test/kotlin` in the project folder.

4.2. Integration Tests

It is your job to also write integration tests to test how the modules work together. In integration tests, you access individual methods similar to unit tests. You can write and execute these tests like **Unit Tests**.

¹<https://www.jacoco.org/>

²<https://junit.org>

³<https://site.mockito.org>

⁴<https://junit.org/junit5/docs/current/user-guide/>

1 4.3. System Tests

- 2 You also need to write system tests that sufficiently test all simulator functionality.
- 3 In order to test the simulator, you have to provide the configuration files for the simulation
- 4 scenario you want to test. Your system tests will also run on the reference implementation,
- 5 so that you can determine if your assumption about the simulation behavior is correct.
- 6 Note that your system tests are also run on faulty simulations (mutants), so your tests must
- 7 find faulty implementations.



Hint for implementation

System tests belong in the given directory `src/systemtest/kotlin` in the project folder.

A. Appendix

A.1. Example recipes and restaurants

Listing A.1: Scenario 2 objects

```

3 {
4   "ingredients": [
5     {"name": "chicken", "unit": "g", "packagingVolume": 1000, "bestBefore": 1},
6     {"name": "rice", "unit": "g", "packagingVolume": 1000, "bestBefore": 3},
7     {"name": "pasta", "unit": "g", "packagingVolume": 1000, "bestBefore": 3},
8     {"name": "garlic", "unit": "g", "packagingVolume": 50, "bestBefore": 1},
9     {"name": "oil", "unit": "mL", "packagingVolume": 500, "bestBefore": 1},
10    {"name": "beef", "unit": "g", "packagingVolume": 100, "bestBefore": 1},
11    {"name": "potato", "unit": "g", "packagingVolume": 1000, "bestBefore": 1},
12    {"name": "onion", "unit": "X", "packagingVolume": 10, "bestBefore": 1}
13  ],
14  "recipes": [
15    {
16      "id": 1, "name": "chicken rice", "duration": 30, "basicDishFor": "ASIAN",
17      "cookType": [EXEC, SOUS, TOURNANT, ROAST], "ingredients": [
18        {"name": "chicken", "unit": "g", "amount": 100}, {"name": "rice", "unit": "g", "amount": 200}, {"name": "onion", "unit": "X", "amount": 1}, {"name": "garlic", "unit": "g", "amount": 10}, {"name": "oil", "unit": "mL", "amount": 20}]
19    },
20    {
21      "id": 2, "name": "beef pasta", "duration": 25, "cookType": [EXEC, SOUS, TOURNANT, ROAST], "ingredients": [
22        {"name": "beef", "unit": "g", "amount": 150}, {"name": "pasta", "unit": "g", "amount": 200}, {"name": "onion", "unit": "X", "amount": 1}, {"name": "garlic", "unit": "g", "amount": 10}, {"name": "oil", "unit": "mL", "amount": 20}]
23    },
24    {
25      "id": 3, "name": "potato soup", "duration": 10, "basicDishFor": "EUROPEAN",
26      "cookType": [EXEC, SOUS, TOURNANT, VEGETABLE], "ingredients": [
27        {"name": "potato", "unit": "g", "amount": 300}, {"name": "onion", "unit": "X", "amount": 1}, {"name": "garlic", "unit": "g", "amount": 10}, {"name": "oil", "unit": "mL", "amount": 20}]
28    }
29  ],
30  "restaurants": [
31    {
32      "id": 1, "name": "Fire", "type": "EUROPEAN",
33      "openingTickStart": 1, "openingTickEnd": 24,
34      "recipes": [1, 2, 3],
35      "kitchenStaff": {"TOURNANT": 1}, "waitstaff": 1, "deliveryDrivers": 3,
36      "tables": [
37        {"id": 2, "type": "COMMON", "size": 9 },
38        {"id": 3, "type": "COMMON", "size": 2 },
39        {"id": 4, "type": "COMMON", "size": 3 }
40      ]
41    }
42  ]

```

```

1      { "id": 5, "type": "COMMON", "size": 3 },
2      { "id": 6, "type": "COMMON", "size": 8 }]],
3      "positiveRatings": 10, "negativeRatings": 5
4  ],
5  "customers": [
6  {
7      "id": 3, "type": "REGULAR", "size": 10, "tableType": "COMMON",
8      "visitingTick": 4, "visitingStart": 20, "visitingPeriod": 10,
9      "restaurant": 1, "foodPreferences": []
10 },
11 {
12     "id": 1, "type": "CASUAL", "size": 8, "tableType": "COMMON",
13     "restaurantTypes": ["EUROPEAN", "AFRICAN"], "deliveryDistance": 0
14     "visitingEvenings": [30, 35], "visitingTick": 1,
15     "foodPreferences": [{"size": 2, "excludedIngredients": ["beef", "potato"]}],
16     "ratingLikelihood": "ALWAYS"
17 },
18 {
19     "id": 2, "type": "CASUAL", "size": 5,
20     "restaurantTypes": ["EUROPEAN", "ASIAN"], "deliveryDistance": 10,
21     "visitingEvenings": [10, 30], "visitingTick": 6,
22     "foodPreferences": [],
23     "ratingLikelihood": "NEVER"
24 },
25 {
26     "id": 5, "type": "CASUAL", "size": 2,
27     "restaurantTypes": ["AMERICAN"], "deliveryDistance": 10,
28     "visitingEvenings": [30, 34], "visitingTick": 6,
29     "restaurantTypes": ["AMERICAN"],
30     "foodPreferences": [],
31     "ratingLikelihood": "SOME"
32 }
33 ]
34 }

```

Listing A.2: Additional objects for scenario 3, the other objects are in [Listing A.1](#)

```

36 {
37     "customers": [
38     {
39         "id": 6, "type": "EVENT", "size": 16, "tableType": "COMMON",
40         "visitingTick": 4, "foodPreferences": [],
41         "restaurantTypes": ["EUROPEAN", "AFRICAN"],
42         "foodPreferences": [{"size": 16, "excludedIngredients": ["chicken", "potato"]}],
43         "eventEvening": 35,
44         "favoriteDishes": {"EUROPEAN": "potato soup"},
45     }
46 ]
47 }
48 }
49 }
50 }

```

¹ A.2. Features

No	Name	Included aspects, definition
F01	Simulation	Managing Main, CLI, and simulation, integrating parsing, evening and tick handling
F02	Logging	Supporting all log messages
F03	Parse Ingredients & Recipes	Parsing and validating ingredients and recipes
F04	Parse Restaurants	Parsing and validating restaurants to just add unit tests
F05	Parse Customers	Parsing and validating customers
F06	Parse Incidents	Parsing and validating incidents
F07	Statistics	Performing simulation statistics
F08	Supplier	Providing ingredients, managing packaging volume
F09	Pantry - Buying Ingredients	Planning dishes, calculating ingredients, restocking
F10	Cooking - Order Queue	Queuing of orders and meals within order, reserving ingredients, deciding next meals to cook
F11	Cooking - Staff & Staff Management	Managing staff id, determining free cooks, assigning cooks to meals to be cooked
F12	Cooking - Cooking	Reserving and managing reserved ingredients, cooking meals, managing finished meals
F13	Restaurant - Menue	Managing available ingredients and available dishes to order
F14	FOH - Evening Table Reservation	Reserving for regular and event tables, canceling reservations
F15	FOH - Table Merging & Tables	Table merging algorithm for all groups
F16	FOH - Seating	Welcoming, assigning waitstaff, seating groups
F17	FOH - Staff Management	Assigning id, managing workload and tick load per staff
F18	FOH - Ordering	Ordering with connection to customers & kitchen
F19	FOH - Serving	Serving meals from kitchen
F20	FOH - Delivery	Bringing meals from kitchen to delivery drivers, managing delivery drivers, restaurant status of drivers
F21	FOH - Escorting	Escorting customers, resetting tables
F22	Customer - Regulars	Performing regular customer behavior, e.g., periodic visit, rating
F23	Customer - Casual	Performing casual customer visiting behavior, e.g., restaurant decision, rating
F24	Customer - Casual Delivery	Performing casual customer delivery behavior, e.g., restaurant decision, accepting food, rating
F25	Customer - Restaurant Decision	Deciding on restaurants for causal groups and casual delivery groups, and event groups
F26	Customer - Food Preferences	Applying food preferences for browsing and ordering
F27	Customer - Waiting for Food	Waiting for food, eating, giving up
F28	Browsing Service	Browsing with static and dynamic restaurant data, estimating available seats
F29	Delivery Service	Receiving meals from kitchen and transporting them, giving them to customers, returning
F30	End of Evening	Aborting, resetting of customers, performing behavior for opening and closing times
F31	Incident - Staff Change	Managing flexible staff storage, performing incident staffchange
F32	Incident - Recipe Change	Managing flexible recipe storage, performing incident receipechange
F33	Incident - Packaging Change	Managing flexible packaging storage, performing incident packagingchange
F34	Incident - Ingredient Unavailability	Managing availability at supplier, performing incident ingredientUnavailability

Table 1: Features of the system

No	Name	Included aspects, definition
P01	Pantry - Best Before, Reservations	Storing ingredients, expiring best before, reserving ingredients for meals correctly
P02	FOH - Event Seating	Assigning waitstaff for event seating
P03	FOH - Event Other Actions	Performing other actions for event groups, like waitstaff selection and action performance
P04	FOH Event -Ordering, Escorting Customer - Events	Performing event customer behavior, e.g., decision, serving, rating
P05	Customer - Rating	Rating an experience, add to rating service, rating service

Table 2: Additional features of the system